

 AVALIAÇÃO

Nome:		R.A.:	
Curso:	Ciência da Computação	Data:	____/____/2026
Disciplina:	Estrutura de Dados	Docente:	Prof. Me. Karan Luciano
Avaliação:	☒ N1 ☐ N2 ☐ Sub ☐ Exame	Nota:	

ANTES DE INICIAR A AVALIAÇÃO, LEIA ATENTAMENTE AS ORIENTAÇÕES ABAIXO:**Composição e Distribuição da Pontuação (Total da Avaliação Escrita: 30,0 pontos):**

- **10 Questões Objetivas (Múltipla Escolha):** valor de **1,5 ponto** cada (totalizando **15,0 pontos**).
- **01 Questão de Análise de Código em C:** valor de **7,5 pontos**.
- **01 Questão Discursiva de Tabela Verdade & Lógica:** valor de **7,5 pontos**.

A pontuação total obtida nesta prova escrita (0 a 30 pontos) compõe a etapa N1 somada às atividades práticas (20 pontos), totalizando os 50 pontos da N1.

1. Confira se este caderno contém todas as **12 questões** (10 objetivas numeradas de 01 a 10 e 02 discursivas/código numeradas de 11 e 12);
2. **Não será permitido o uso do telefone celular ou qualquer outro dispositivo eletrônico. Devem ser desligados e guardados;**
3. As questões objetivas contêm apenas uma resposta correta. A marcação de mais de uma resposta invalidará a questão;
4. Transcreva as respostas das questões objetivas no Gabarito abaixo, com caneta azul ou preta e sem rasuras;
5. As respostas das questões discursivas e de código devem ser redigidas à caneta no espaço delimitado;
6. **Não será permitido nenhum tipo de consulta a livros, anotações ou colegas;**
7. O tempo mínimo de permanência em sala de aula é de 30 minutos. Os últimos 03 alunos devem permanecer até o término da prova por estes.

GABARITO — QUESTÕES OBJETIVAS

Marque apenas as questões correspondentes à sua avaliação com um 'X'. Rasuras invalidam a resposta.

01	A	B	C	D	E	06	A	B	C	D	E
02	A	B	C	D	E	07	A	B	C	D	E
03	A	B	C	D	E	08	A	B	C	D	E
04	A	B	C	D	E	09	A	B	C	D	E
05	A	B	C	D	E	10	A	B	C	D	E

CADERNO DE QUESTÕES OBJETIVAS

QUESTÃO 01

Ponteiros e Gestão de Memória (1,5 pt)

Em linguagem C, a memória física (RAM) pode ser compreendida didaticamente como uma sequência de compartimentos numerados (endereços de memória). Ao declararmos no código a instrução `int x = 42; int *ptr = &x;`, qual é a finalidade precípua da variável `ptr` na arquitetura de execução?

- A) Guardar o endereço de memória física onde a variável referenciada está armazenada.
- B) Duplicar automaticamente o valor de `x` em uma área reservada de backup da memória.
- C) Executar a limpeza e liberação automática do espaço ocupado por `x` no término do bloco.
- D) Armazenar valores numéricos fracionários de ponto flutuante de dupla precisão.
- E) Converter o valor decimal de `x` em um tipo caractere imutável.

QUESTÃO 02

Operadores de Indireção (1,5 pt)

Considere a execução sequencial do seguinte trecho de código em linguagem C:

```
int numero = 10;
int *ptr = &numero;
*ptr = 25;
```

Após o processamento dessas três instruções, qual será o valor impresso pela instrução `printf("%d", numero);` e qual mecanismo fundamenta essa alteração?

- A) Imprime **10**, pois ponteiros operam apenas sobre cópias locais isoladas em registradores.
- B) Imprime **25**, pois o operador de desreferenciação (`*`) altera diretamente a posição de memória original de `numero`.
- C) Imprime o endereço hexadecimal da variável `numero` na pilha de execução (*Stack*).
- D) Imprime **0**, pois a atribuição via ponteiro reinicializa o conteúdo da variável apontada.
- E) Ocorre um erro de violação de acesso (*Segmentation Fault*) por atribuição ilegal de tipo inteiro a ponteiro.

QUESTÃO 03

Segurança e Ponteiro Nulo (1,5 pt)

Um acadêmico escreveu a seguinte instrução em seu programa:

```
int *ptr;
*ptr = 50;
```

Ao compilar e executar o programa, a aplicação foi encerrada abruptamente pelo sistema operacional com uma falha de segmentação (*Segmentation Fault*). Qual o motivo técnico dessa falha e qual a boa prática recomendada para evitar ponteiros soltos (*wild pointers*)?

- A) A variável ponteiro precisa ser obrigatoriamente do tipo `float` para receber atribuições numéricas.
- B) O ponteiro `ptr` não foi inicializado e continha um endereço arbitrário (lixo de memória), violando o espaço protegido; a boa prática é inicializá-lo com o endereço de uma variável real com `&` ou com a constante `NULL`.
- C) O operador `*` foi esquecido na declaração do ponteiro, impossibilitando a operação de escrita.
- D) Em linguagem C, é proibido atribuir números inteiros a ponteiros, sendo permitido apenas caracteres alfanuméricos.
- E) A falha ocorreu porque a função `printf` não foi chamada imediatamente antes da atribuição.

QUESTÃO 04

Passagem de Parâmetros em Funções (1,5 pt)

Deseja-se construir uma função em C que dobre o valor de uma variável declarada na função `main`, de modo que o resultado persista após o término da função. Qual protótipo e chamada atendem corretamente à passagem por referência?

- A) Protótipo: `void dobrar(int n);` | Chamada: `dobrar(numero);`
- B) Protótipo: `void dobrar(int *n);` | Chamada: `dobrar(&numero);`
- C) Protótipo: `int dobrar(int &n);` | Chamada: `dobrar(*numero);`
- D) Protótipo: `void dobrar(int n*);` | Chamada: `dobrar(numero&);`
- E) Protótipo: `void dobrar(&int n);` | Chamada: `dobrar(&numero);`

QUESTÃO 05

Lógica Proposicional (1,5 pt)

Na Lógica Proposicional, uma proposição é definida como uma oração declarativa que exprime um pensamento de sentido completo e à qual se pode atribuir unicamente um de dois valores lógicos: Verdadeiro (V) ou Falso (F), sem ambiguidade. Dadas as quatro sentenças abaixo:

- I. A linguagem C foi criada por Dennis Ritchie."
- II. Qual é o valor guardado no endereço de memória?"
- III. Um ponteiro inicializado com NULL não aponta para nenhuma região válida de memória."
- IV. Feche imediatamente o editor de texto!"

São formalmente consideradas **proposições lógicas**:

- A) Apenas as sentenças I e III.
- B) Apenas as sentenças II e IV.
- C) Apenas a sentença I.
- D) Todas as sentenças (I, II, III e IV).
- E) Apenas as sentenças I, II e III.

QUESTÃO 06

Tabela Verdade e Condicional (1,5 pt)

Na tabela verdade de uma proposição condicional (implicação) $P \rightarrow Q$, o valor lógico resultante é **Falso** unicamente quando:

- A) O antecedente P é Falso e o consequente Q é Verdadeiro ($F \rightarrow V$).
- B) O antecedente P e o consequente Q são ambos Falsos ($F \rightarrow F$).
- C) O antecedente P é Verdadeiro e o consequente Q é Falso ($V \rightarrow F$).
- D) O antecedente P e o consequente Q são ambos Verdadeiros ($V \rightarrow V$).
- E) Qualquer uma das duas proposições individuais assumir valor Falso.

QUESTÃO 07

Álgebra Proposicional e Operadores em C (1,5 pt)

Para construir a tabela verdade completa de uma expressão lógica contendo 3 variáveis proposicionais independentes (P , Q e R), a quantidade exata de linhas necessárias e os respectivos operadores correspondentes na linguagem C para Conjunção (E) e Disjunção Inclusiva (OU) são:

- A) 6 linhas; operadores `&` e `|`.
- B) 8 linhas (2^3); operadores `&&` e `||`.
- C) 9 linhas (3^2); operadores `AND` e `OR`.
- D) 8 linhas (2^3); operadores `=` e `!=`.
- E) 16 linhas (2^4); operadores `&&` e `||`.

QUESTÃO 08

Controle de Versão com Git (1,5 pt)

O Git organiza o ciclo de vida dos arquivos em três áreas/estados fundamentais na máquina de desenvolvimento. Ao editar um arquivo em C no editor e preparar essa alteração para fazer parte do próximo *commit*, o arquivo transita, respectivamente, por quais áreas do Git?

- A) Do *Working Directory* (diretório de trabalho) para a *Staging Area* (área de preparação / index).
- B) Do *Remote Repository* (GitHub) diretamente para o *Commit History*.
- C) Da *Staging Area* diretamente para a nuvem remota sem necessidade de confirmação.
- D) Da memória RAM do sistema operacional diretamente para o arquivo binário compilado.
- E) Da lixeira do sistema de arquivos para o *Local Repository*.

QUESTÃO 09

Comandos do Git (1,5 pt)

Um desenvolvedor alterou o arquivo `ponteiros.c` e executou no terminal o comando `git add ponteiros.c`. Sobre a situação atual do arquivo no repositório, assinale a afirmação correta:

- A) A alteração já está definitivamente registrada como um novo ponto no histórico do Git.
- B) O arquivo foi enviado imediatamente para o servidor remoto no GitHub.
- C) O arquivo foi apenas colocado no palco de preparação (*Staging Area*); para consolidar permanentemente a alteração no histórico local é necessário executar `git commit -m "mensagem"`.
- D) O comando apagou as alterações locais e restaurou a versão anterior salva no último commit.
- E) O Git inicializou um novo repositório vazio naquele diretório específico.

QUESTÃO 10*Repositórios Remotos e Colaboração (1,5 pt)*

No trabalho em equipe com Git e GitHub, a sincronização entre a máquina local do desenvolvedor e o repositório centralizado na nuvem baseia-se nos comandos **git push** e **git pull**. Suas funções são, respectivamente:

- A) **git push** baixa atualizações da nuvem para o computador; **git pull** envia commits locais para a nuvem.
- B) **git push** envia os commits do repositório local para o GitHub; **git pull** baixa e mescla os commits do GitHub no repositório local.
- C) **git push** descarta alterações não salvas; **git pull** inicializa a branch principal (*main*).
- D) Ambos os comandos executam a mesma operação de criação de chaves criptográficas SSH.
- E) **git push** compila o código em C; **git pull** executa o depurador de memória.

QUESTÕES DISCURSIVAS E ANÁLISE DE CÓDIGO

QUESTÃO 11 (ANÁLISE DE CÓDIGO)

Ponteiros, Passagem por Referência e Teste de Mesa (7,5 pts)

Um acadêmico de Ciência da Computação implementou o programa abaixo em linguagem C para simular o comportamento de ponteiros e passagem de parâmetros por referência na memória RAM:

```
#include <stdio.h>

void atualizar(int *a, int *b) {
    int aux = *a;
    *a = *b + 5;
    *b = aux * 2;
}

int main() {
    int x = 10;
    int y = 20;
    int *p1 = &x;
    int *p2 = &y;

    atualizar(p1, p2);

    printf("x = %d, y = %d ", x, y);
    return 0;
}
```

a) (3,75 pts) Apresente a **saída exata** impressa no console pela instrução **printf** contida na função **main** ao término da execução do programa. Demonstre o cálculo passo a passo através do **Teste de Mesa** acompanhando os valores de **x**, **y**, ***a**, ***b** e **aux** na memória RAM.

b) (3,75 pts) Explique tecnicamente por que os valores das variáveis **x** e **y** foram modificados permanentemente no escopo da função **main**. O que teria acontecido com os valores de **x** e **y** se a assinatura da função fosse **void atualizar(int a, int b)** e a chamada fosse **atualizar(x, y)**? Diferencie com precisão a passagem por valor da passagem por referência.

QUESTÃO 12 (DISCURSIVA)

Tabela Verdade, Conectivos e Classificação (7,5 pts)

No desenvolvimento de sistemas computacionais críticos, expressões lógicas compostas são frequentemente empregadas em estruturas de decisão (if/else). Considere a proposição composta expressa por:

$(P \wedge \neg Q) \rightarrow P$

a) (4,50 pts) Construa e preencha a **tabela verdade completa** da proposição para todas as combinações possíveis de valores lógicos das variáveis *P* e *Q*, resolvendo passo a passo cada coluna intermediária:

P	Q	¬Q	P ∧ ¬Q	(P ∧ ¬Q) → P
V	V			
V	F			
F	V			
F	F			

b) (3,00 pts) Com base no resultado obtido na última coluna da tabela verdade, classifique formalmente essa proposição composta como uma **Tautologia**, **Contradição** ou **Contingência**. Justifique tecnicamente a sua resposta definindo com rigor o conceito da classificação adotada.